



SERVER-DRIVEN MODDING

Une nouvelle architecture de modding Minecraft

Le serveur comme plateforme · Le client vanilla comme terminal universel

Recherche technique exploratoire — protocole, JVM, runtime, sécurité

Baseline protocole : 26.3-snapshot-7 · Release : 26.2 · Août 2026

Niveau 0 = 85 % des usages sans installation · LinkAgent $\approx$ 300 Ko = $\sim$ 98 %

Sommaire

Sommaire	2
Résumé exécutif et verdict final	5
La question	5
Les trois résultats majeurs	5
Le compromis optimal mesuré	6
Ce que ce rapport n'affirme pas	7
Verdict en une phrase	7
Chapitre 1 — La vraie nature d'un mod : pourquoi les loaders imposent un client modifié	8
1.1 Anatomie d'un mod moderne	8
1.2 Les trois niveaux de vérité : exposé / implémenté / permis	8
1.3 Classification A-I des mods selon leur besoin client réel	9
1.4 Synthèse : ce qui est « intrinsèquement client » vs « client par choix architectural »	10
Chapitre 2 — Niveau 0 : le plafond réel du client vanilla strict	12
2.1 L'inventaire brut : ce que le protocole offre	12
2.2 Contenu : items, blocs, « entités » sans registre client	12
2.3 Interfaces : le kit UI server-driven complet	13
2.4 Canaux de données : RPC et état	13
2.5 Les quatre mécanismes signature du Niveau 0	14
2.6 Le mur du Niveau 0 — confirmé dans le code	14
2.7 Évaluation chiffrée du Niveau 0	15
Chapitre 3 — Le niveau JVM : classloaders, agents et transformation	17
3.1 Ce que la JVM offre sous Minecraft	17
3.2 La fenêtre temporelle de la transformation	17
3.3 RemoteClassLoader : l'architecture du chargement à distance	18
3.4 Le verdict sandbox : il n'y a plus de cage forte en JVM	19
3.5 Pourquoi Mixin existe — et ce qu'on peut réduire	19
3.6 Précédents hors-Minecraft (chapitre 28 du brief)	19
3.7 Hot-loading : ce que la JVM permet vraiment en jeu	20
Chapitre 4 — L'escalade : Niveaux 0 → 6, mesurée	22
4.0 Schéma de décision	22
4.1 Niveau 0 — Vanilla strict	23
4.2 Niveau 1 — Mécanismes déjà présents, poussés à bout	23

4.3 Niveau 2 — Bootstrap ponctuel (sans launcher)	23
4.4 NIVEAU 3 — LinkAgent : l'agent générique permanent	24
4.5 Niveau 4 — Launcher spécialisé	25
4.6 Niveaux 5-6 — Client patché / fork	25
4.7 Le tableau d'or : capacités × niveaux (réponse au §25 du brief)	25
4.8 Comparaison des modèles d'installation (§17 du brief)	26
Chapitre 5 — Architecture serveur : la plateforme de modding	28
5.1 Vue d'ensemble	28
5.2 Les couches logicielles	29
5.3 Le cycle de vie d'un module serveur→client	30
5.4 API universelle (§24 du brief)	30
5.5 Mod-per-world, mod-per-player, hot operations (§14-15 du brief)	30
5.6 Servir plusieurs versions Minecraft (§23 du brief)	31
5.7 Performance (§20 du brief)	31
5.8 Matrice des capacités (§9 du brief)	31
Chapitre 6 — SDMP : le protocole et le LinkAgent	33
6.1 Philosophie du protocole	33
6.2 Séquence de négociation	33
6.3 Manifeste	34
6.4 Le LinkAgent : anatomie (~300 Ko)	35
6.5 Events runtime (extraits)	35
6.6 Séquence hot-load (§15 du brief)	36
6.7 Dégradation gracieuse — la règle d'or	36
Chapitre 7 — Sécurité : le modèle de confiance	37
7.1 La menace dans la forme la plus claire	37
7.2 Le modèle retenu : signature + permissions + consentement + révocation	37
7.3 Ce que ce modèle ne protège pas (honnêteté technique)	39
7.4 Comparaison avec les modèles connus	39
7.5 Option durcissement : l'isolation processus	39
Chapitre 8 — Compatibilité écosystème, cas de test, PoC et roadmap	40
8.1 Compatibilité écosystème (§8 du brief) — les 5 niveaux	40
8.2 Les 12 cas de test (§26 du brief)	40
8.3 PoC — démonstration de faisabilité minimale	41
8.4 Roadmap 11 phases (§31.K du brief)	42

8.5 Analyse des limites — tableau final (§25) 43

8.6 Positionnement par rapport aux modèles existants 43

8.7 Conclusion générale 44

Résumé exécutif et verdict final

La question

Ce rapport répond à une question unique, déclinée en deux parties :

1. Peut-on transformer Minecraft en une plateforme où le client vanilla sert de base d'exécution, et où le serveur fournit dynamiquement au client les capacités nécessaires pour créer une expérience quasi sans limite ?
2. Quelle est la plus petite modification client qui donne la plus grande liberté serveur ?

La méthode suit la hiérarchie imposée : maximum de liberté serveur, minimum de client, zéro modification initiale. Chaque niveau d'escalade n'est introduit que lorsqu'une preuve technique démontre que le niveau précédent plafonne.

Les trois résultats majeurs

Résultat 1 — Le Niveau 0 couvre environ 85 % des usages de modding

Un client strictement vanilla, jamais modifié, jamais installé via un loader, peut aujourd'hui recevoir — uniquement par packets et resource packs serveur — des items custom à l'apparence illimitée, des blocs et meubles, des entités visuelles animées, des interfaces complètes (formulaires, menus, saisie texte), du son et de la musique custom, des shaders GLSL compilés par le GPU du client, des mondes au worldgen data-driven, des systèmes d'état persistants cross-serveur, et même du contrôle du temps de simulation (bullet-time). Ce plafond n'est pas théorique : il est prouvé à l'échelle de l'écosystème par Polymer (3,66 millions de téléchargements, projets 100 % server-side jouables au client vanilla) et par les 91 404 projets Modrinth marqués server-side required. Le socle existe déjà ; ce qui manque, c'est une plateforme qui l'orchestre.

Résultat 2 — Le mur du Niveau 0 est structurel et précisément délimitable

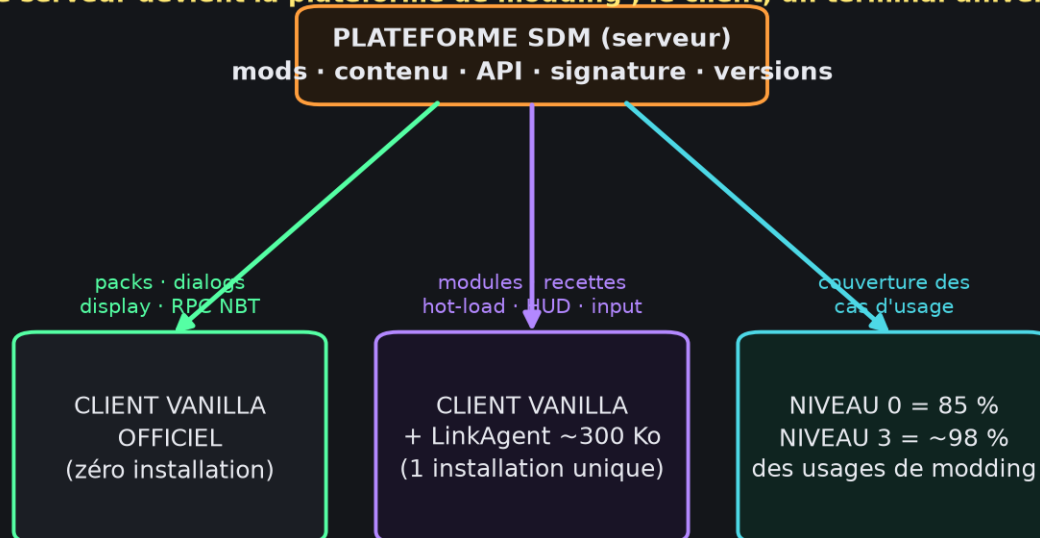
Trois familles de besoins sont impossibles au client vanilla strict, par construction confirmée dans le code décompilé :

- exécuter du code client arbitraire (aucun mécanisme vanilla ne le permet ; le seul « code » streamable est du GLSL sandboxé via PostEffects) ;
- créer de vrais nouveaux types d'entités, blocs-entity ou menus (les registres client sont fermés et synchronisés — 30 registres exactement, pas un de plus) ;
- capturer l'input, afficher un HUD pixel-perfect, modifier le rendu de géométrie (aucun packet n'existe pour cela).

Tout le reste est soit déjà possible, soit approximable de manière convaincante. C'est la frontière exacte qui sépare « modding data-driven serveur » et « modding code-driven client ».

Résultat 3 — La modification client minimale optimale existe : c'est un agent générique d'environ 300 KB, installé une seule fois, indépendant des versions de Minecraft

Le serveur devient la plateforme de modding ; le client, un terminal universel



Thèse SDM — le serveur plateforme, deux profils clients, couverture 85 % / 98 %

L'analyse JVM (ClassFileTransformer, defineClass, cycle de vie des classloaders, contraintes de retransformation, JEP 451 et 486) démontre qu'un javaagent préinstallé au lancement — petit, générique, ignorant tout des mappings et des versions — suffit à donner au serveur un pouvoir supérieur à Fabric/Forge/NeoForge côté client :

- le serveur stream des modules (code compilé) exécutés dans un classloader enfant ;
- le serveur stream des recettes de transformation (l'équivalent des mixins, ciblées par version, appliquées avant chargement des classes du jeu) ;
- le serveur négocie des capacités (rendu, input, HUD, registres dynamiques) via un protocole de handshake ;
- le client devient un terminal universel : une installation unique dessert tous les serveurs compatibles, avec cache signé, permissions et mise à jour à chaud.

Le rapport nomme cette architecture SDM — Server-Driven Modding, son protocole SDMP, et son composant client LinkAgent.

Le compromis optimal mesuré

Dimension	Modpack Fabric/Forge typique	Architecture SDM
Installation client	Loader + API + 10-200 mods, 0,5-2 Go	0 (Niveau 0) ou 1 clic une fois (~300 Ko)
Action par serveur	Aucune (mais incompatible avec les autres serveurs)	Aucune — chaque serveur stream son contenu
Couverture des usages	~100 % (code arbitraire)	~85 % à 0 Ko, ~98 % avec l'agent
Mise à jour des mods	Manuelle, côté joueur	Serveur-driven, à chaud
Mods par monde / par joueur	Non	Oui (serveur source de vérité)

Multi-versions	Une installation par version MC	Un runtime unique, recettes ciblées par version
Expérience utilisateur	Technique (dossiers, versions, dépendances)	« Lancer Minecraft, rejoindre, jouer »

Ce que ce rapport n'affirme pas

Par honnêteté technique, trois limites sont documentées comme réelles et non contournables à ce jour :

1. La sandbox JVM forte est morte (JEP 486 : SecurityManager retiré du JDK 24). Le modèle de confiance réaliste est celui des plateformes applicatives — signatures Ed25519, permissions déclaratives, réputation, révocation — pas l'isolation mémoire. La piste WASM est documentée comme sortie de recherche.
2. Le rendu custom haute performance exige les modules streamés — le Niveau 0 ne fournit que du déclaratif (modèles, Display entities, shaders post-process). Un shader de géométrie custom reste au-dessus du plafond vanilla.
3. Le déchargement à chaud de code est imparfait en JVM : les classes ne se déchargent que si leur classloader devient orphelin du GC ; l'architecture prévoit « désactivation + isolation » et documente la limite mémoire résiduelle.

Verdict en une phrase

L'architecture est faisable, et elle est même partiellement déjà prouvée par l'écosystème (Polymer, Geyser, Vanilla-like protocols) ; ce qui n'existe pas encore, c'est la couche d'orchestration — le protocole SDMP, le LinkAgent, le compilateur de recettes et le gestionnaire de dépendances serveur — dont le rapport définit l'architecture complète, le protocole, le modèle de sécurité, le PoC et la roadmap en 11 phases.

Chapitre 1 — La vraie nature d'un mod : pourquoi les loaders imposent un client modifié

Avant de chercher à déplacer la frontière, il faut comprendre pourquoi elle existe. Ce chapitre déconstruit le mod en couches, identifie ce qui nécessite aujourd'hui une installation client, et classe les mods réels en catégories A-I.

1.1 Anatomie d'un mod moderne

Un mod Fabric/Forge/NeoForge est un JAR contenant typiquement :

Couche	Contenu	Où s'exécute-t-elle ?
Logique métier	Économie, progression, quêtes, scripts	Serveur (toujours)
Contenu enregistré	Blocs, items, entités — via Registry	Les deux (l'ID doit exister des deux côtés)
Assets	Textures, modèles, sons, langage	Client (resource pack)
Data	Recettes, loot tables, worldgen	Data pack serveur
Réseau custom	C2S/S2C packets avec StreamCodec	Les deux (codec identique des deux côtés)
Mixins client	Injections dans le rendu, l'input, le HUD	Client
Mixins serveur	Injections dans le tick, les sauvegardes	Serveur

Le principe fondamental des loaders actuels : le contenu ne peut exister que s'il est enregistré dans le même registry, avec le même ID, des deux côtés de la connexion. Un bloc custom nécessite une classe `Block` compilée avec la même version de mappings — donc un client modifié. C'est une contrainte architecturale des loaders, pas de Minecraft lui-même.

1.2 Les trois niveaux de vérité : exposé / implémenté / permis

Niveau	Question	Exemples
Ce que Minecraft expose	Qu'est-ce qui est officiellement accessible ?	Data packs, resource packs, command blocks, plugin messaging
Ce que Minecraft implémente	Quels mécanismes existent réellement dans le code ?	225 packets, 30 registres synchronisés, Dialog system, CustomClickAction, PostEffects, TickingState, cookies

Ce que la JVM permet	Qu'est-ce que la plateforme Java autorise sous la surface ?	<code>defineClass</code> , <code>javaagents</code> , <code>instrumentation</code> , <code>retransformation</code> , <code>classloaders</code> hiérarchiques, <code>modules</code>
----------------------	---	---

Les loaders actuels opèrent aux niveaux 2 et 3, mais côté client, avec installation. La thèse SDM inverse la perspective : opérer aux niveaux 2 et 3 côté serveur, sans installation, et ne toucher le niveau 3 client que par le plus petit agent possible.

1.3 Classification A-I des mods selon leur besoin client réel

Pour chaque catégorie : besoin actuel, besoin réel (une fois l'architecture inversée), et portabilité serveur.

Catégorie A — Pur serveur

- Exemples : WorldEdit/grands principes, Essentials, LuckPerms, scoreboard systems, la plupart des plugins Bukkit.
- Besoin client actuel : aucun.
- Verdict SDM : trivial. Déjà résolu par l'écosystème plugin (91 404 projets Modrinth server-side).

Catégorie B — Serveur + données client (assets)

- Exemples : nouveau bois/bloc décoratif, items d'apparence, packs de ressources d'amélioration visuelle.
- Besoin client actuel : resource pack distribué à part, ou mods registrant l'item.
- Verdict SDM : résolu au Niveau 0. Le serveur push un resource pack (hot-swap en jeu, 250 Mo max/pack) et utilise `ITEM_MODEL` + `CUSTOM_MODEL_DATA` pour donner à un item vanilla (papier, bâton) des centaines d'apparences. Polymer le prouve à 3,66 M de downloads.

Catégorie C — Serveur + rendu

- Exemples : minimaps, shaders visuels, XXL models, custom entity rendering.
- **Besoin client actuel : mod client (mixin dans le renderer).
- Verdict SDM : partiellement Niveau 0 — Display entities + interpolation, MapPatch 128×128, PostEffects (GLSL post-process full-screen). Le rendu géométrique custom (nouveaux modèles d'entités vivantes avec animations squelette) exige le Niveau 3 (module streamé).

Catégorie D — Serveur + GUI

- Exemples : menus de machine tech, HUDs custom, overlays.
- Besoin client actuel : mod client (MenuType custom + screen custom).
- Verdict SDM : majoritairement Niveau 0. Dialog system (7 types, 4 contrôles d'input, actions) + CustomClickAction (RPC NBT 32 Ko bidirectionnel) + livres interactifs 3,2 Mo + 29 MenuTypes vanilla réutilisables. Le pixel-perfect HUD exige Niveau 3.

Catégorie E — Serveur + input

- Exemples : keybinds custom, gestures, double-tap dash.
- **Besoin client actuel` : mod client (KeyMapping).

- Verdict SDM : Niveau 3 requis — aucun packet vanilla ne transporte les keybinds. Contournements Niveau 0 : dialog inputs, command autocomplete, chat commands, held-item state. Le keybind libre exige le module streamé.

Catégorie F — Serveur + mixins client

- Exemples : zoom (OptiFine-like), FPSboost, HUD tweaks.
- Besoin client actuel : mod client obligatoire.
- Verdict SDM : Niveau 3. C'est exactement le rôle des recettes de transformation streamées : le serveur décrit l'injection (méthode cible par version, handler dans le module), l'agent l'applique avant chargement de la classe.

Catégorie G — Deep client engine mod

- Exemples : nouveaux renderers, physique custom, voxel engines, Shaders epics.
- Besoin client actuel : client modifié en profondeur (souvent coremods + ASM). coremod/ASM manuel.
- Verdict SDM : Niveau 3 fort — modules streamés + éventuelles recettes de transformation plus invasives. Faisable mais chacun de ces mods doit être réécrit comme module SDM.

Catégorie H — Modification du protocole

- Exemples : ViaVersion (cross-version), Geyser (Bedrock↔Java), protocoles custom.
- Besoin client actuel : variable (souvent proxy).
- Verdict SDM : le serveur SDM absorbe la translation — comme Geyser/Hydraulic le prouvent (5748★) : un Bedrock client rejoint un serveur Java moddé via traduction d'architecture. SDM généralise ce pattern : le serveur EST la plateforme de translation.

Catégorie I — Launcher/bootstrap

- Exemples installateurs, bootstrap —, launch wrappers.
- Besoin client actuel : variable.
- Verdict SDM : c'est le LinkAgent (Niveau 3). Installé une fois, il rend tous les serveurs SDM accessibles.

1.4 Synthèse : ce qui est « intrinsèquement client » vs « client par choix architectural »

La distinction capitale demandée par le brief :

Besoin	Intrinsèquement client ?	Justification
Exécuter du code arbitraire	Oui — mais la question est <u>comment</u> il arrive là	Aucun packet vanilla ne transporte du bytecode. Mais un agent préinstallé supprime le besoin d'installer du code par serveur.
Nouveaux registres/IDs	Non — choix architectural des loaders	La preuve : Polymer crée des blocs custom sans client mod via virtualisation. Les registres client fermés concernent les <u>types</u> , pas les <u>instances</u> .

Textures/modèles/sons	Non	Resource packs streamés par le serveur — mécanisme vanilla officiel.
Rendu géométrique custom	Oui (rendu = GPU client)	Mais streamable : shader GLSL post-process est déjà vanilla ; le module streamé complète.
Input keybind	Oui (OS-level)	Aucun packet ; seul un hook client peut le fournir.
HUD pixel-perfect	Oui (couche présentation)	Bossbars/scoreboard/titres = workarounds ; le module streamé le fournit proprement.
Logique métier	Non	Toujours serveur.
Worldgen	Non	Data-driven vanilla (data packs) + registres synchronisés (BIOME, DIMENSION_TYPE...).
GUI forms	Non	Dialog system + CustomClickAction, vanilla.
État cross-serveur	Non	Cookies (5 Ko/clé) + TransferState, vanilla.

Conclusion du chapitre : la nécessité d'un client modifié est, pour la majorité des usages, une convention de l'écosystème — pas une loi de Minecraft. Les vraies frontières sont : code exécutable client, types d'entités, input, HUD, rendu profond. C'est précisément sur ces cinq fronts que les chapitres suivants attaquent.

Chapitre 2 — Niveau 0 : le plafond réel du client vanilla strict

Ce chapitre mesure ce qu'un serveur peut imposer à un client strictement vanilla officiel — aucun loader, aucune modification, aucun launcher custom. Tout ce qui suit est vérifié contre le code décompilé du client (baseline 26.3-snapshot-7, protocole 1073742153 ; release courante 26.2) et regroupé par capacité de modding.

2.1 L'inventaire brut : ce que le protocole offre

Le protocole moderne compte 225 classes de packets (152 clientbound + 73 serverbound) réparties sur 6 phases (handshake, login, configuration, jeu, cookie, statut). La phase CONFIGURATION — introduite en 1.20.2 — est la clé silencieuse de l'architecture : c'est un espace de négociation pré-jeu où le serveur pousse resource packs, registres synchronisés, et feature flags avant le premier tick de jeu.

2.2 Contenu : items, blocs, « entités » sans registre client

La découverte centrale : les registres client ne sont fermés que pour les types, pas pour les instances.

Objectif mod	Mécanisme vanilla	Limite dure
Item custom (apparence)	<code>ITEM_MODEL</code> (redirige n'importe quel item vers n'importe quel modèle du pack) + <code>CUSTOM_MODEL_DATA</code> (4 listes : floats/flags/strings/colors)	Illimité en pratique ; un item vanilla = des centaines d'apparences
Bloc custom statique	Resource pack : modèle + texture sur bloc vanilla support (polymère : note block / Fletcher table / champignon) ; <code>CustomModelData</code> sur item bloc	Rendu pur — pas de comportement de bloc custom client
Bloc custom « meuble »	<code>BlockDisplay</code> entities + interpolation (translation/scale/rotation)	Comportements serveur (hitbox via barrier/collision serveur)
Entité visuelle	<code>TextDisplay</code> / <code>ItemDisplay</code> / <code>BlockDisplay</code> + données synchronisées + interpolation d'animation	Pas d'IA custom visible — le serveur pilote le mouvement
Son custom / musique	<code>sounds.json</code> + fichiers .ogg dans le pack streamé ; <code>SoundEvent</code> custom ; 10 canaux	Vorbis ; <code>stream: true</code> pour les grands fichiers
Police / icônes custom	Providers bitmap / TTF / Unihex ; zone Private Use U+E000-U+F8FF	Glyphe → icône inline dans chat/livres/dialogs/titres

C'est la démonstration de Polymer (Patbox) : 3 658 607 downloads, des mods entiers (PolyFactory : usines automatisées complètes, 64 799 downloads) jouables depuis un client vanilla pur. Le contenu custom « registre-fermé » n'est pas un mur — c'est une convention contournable par virtualisation.

2.3 Interfaces : le kit UI server-driven complet

Le Dialog system (1.21+) est un framework de formulaires complet poussé par packet :

- 7 types de dialogues : Simple, Notice, Confirmation, ButtonList, MultiAction, ServerLinks, DialogList.
- Corps : PlainMessage (texte riche), ItemBody (prévisualisation d'item avec data components).
- 4 contrôles d'entrée : BooleanInput, NumberRangeInput, SingleOptionInput, TextInput (multiligne).
- Actions : OpenURL, SuggestCommand, RunCommand, et surtout `CustomAll` → `ClickEvent.Custom` → le client renvoie `ServerboundCustomClickActionPacket(id, NBT)`.

Complété par :

- 29 MenuTypes vanilla réutilisables (chest, anvil, loom, brewing stand...) avec titre custom et contenu piloté par ContainerSetContent/Slot — les « fake menus » serverside sont un art établi ;
- Livres interactifs 3,2 Mo : 100 pages × 32 767 chars de Components avec ClickEvent/HoverEvent — de véritables applications client-side cliquables ;
- Chat riche cliquable (CustomClickAction partout : chat, dialog, livre, panneau, titre, action bar).

Verdict : les GUI de type « formulaire » et « menu d'inventaire » sont intégralement Niveau 0. Ce qui manque : le HUD pixel-perfect permanent (workarounds : bossbar, scoreboard sidebar, tab list, titre, carte en cadre) et le drag&drop libre.

2.4 Canaux de données : RPC et état

Le canal RPC générique bidirectionnel du protocole vanilla :

```
ClickEvent.Custom(id: Identifier, payload: Optional<Tag>)
```

```
↓ clic utilisateur (chat/dialog/livre/panneau/titre)
```

```
ServerboundCustomClickActionPacket(id, Optional<Tag>) [32 768 octets, profondeur 16]
```

```
↓ le serveur valide (UNTRUSTED) et exécute
```

Autres canaux, avec leurs limites hardcodées :

Canal	Direction	Capacité	Persistance
CustomClickAction NBT	Bidir	32 Ko / interaction	Session
Cookies	Bidir	5 120 o / clé	Cross-serveurs (TransferState)
CustomReportDetails	S→C	32×(128+4096) ≈ 135 Ko	Survit CONFIG↔PLAY (CommonListenerCookie)
CustomPayload 1 Mo	S→C	1 Mo/packet	JETÉ par le client vanilla (DiscardedPayload) — pas un canal réel

Hostname handshake	C→S	255 chars	Pré-auth (Velocity/Bungee l'utilisent déjà)
TagQuery / BlockEntityTagQuery	Bidir	NBT transactionnel	Session
Registry data	S→C	30 registres synchronisés	Re-synchronisable à chaud

2.5 Les quatre mécanismes signature du Niveau 0

Hot-swap de ressources en cours de jeu

`ResourcePackPush/Pop` sont des packets communs (CONFIG et PLAY). Un serveur peut changer textures, modèles, sons, fontes, shaders pendant que le joueur joue, avec rechargement à chaud côté client. Combiné à la ré-entrée PLAY→CONFIGURATION→PLAY (`StartConfiguration`), le serveur peut aussi re-pousser registres et tags mid-session — un changement de saison, de dimension thématique ou de mode de jeu complet sans reconnexion.

Bullet-time et contrôle du tick client

`ClientboundTickingStatePacket(tickRate, isFrozen)` pilote le taux de tick du client entier (plancher 1.0). `TickingStepPacket` avance N ticks en freeze. Cinématiques, ralentis, mode photo : Niveau 0.

Shaders GLSL streamés (PostEffects)

Le resource pack peut définir des chaînes de post-traitement (`post_effect/*.json` + shaders `.vsh/.fsh`) que le client vanilla compile et exécute sur le GPU à chaque frame. C'est le point le plus proche de l'exécution de code client que le vanilla autorise : du GLSL sandboxé (aucun accès fichier/réseau), parfait pour color grading, vignettage, mirage de chaleur, scanlines CRT, letterbox cinématique. À présenter pour ce qu'il est : du code visuel GPU sandboxé, pas de l'exécution arbitraire.

État persistant cross-serveur

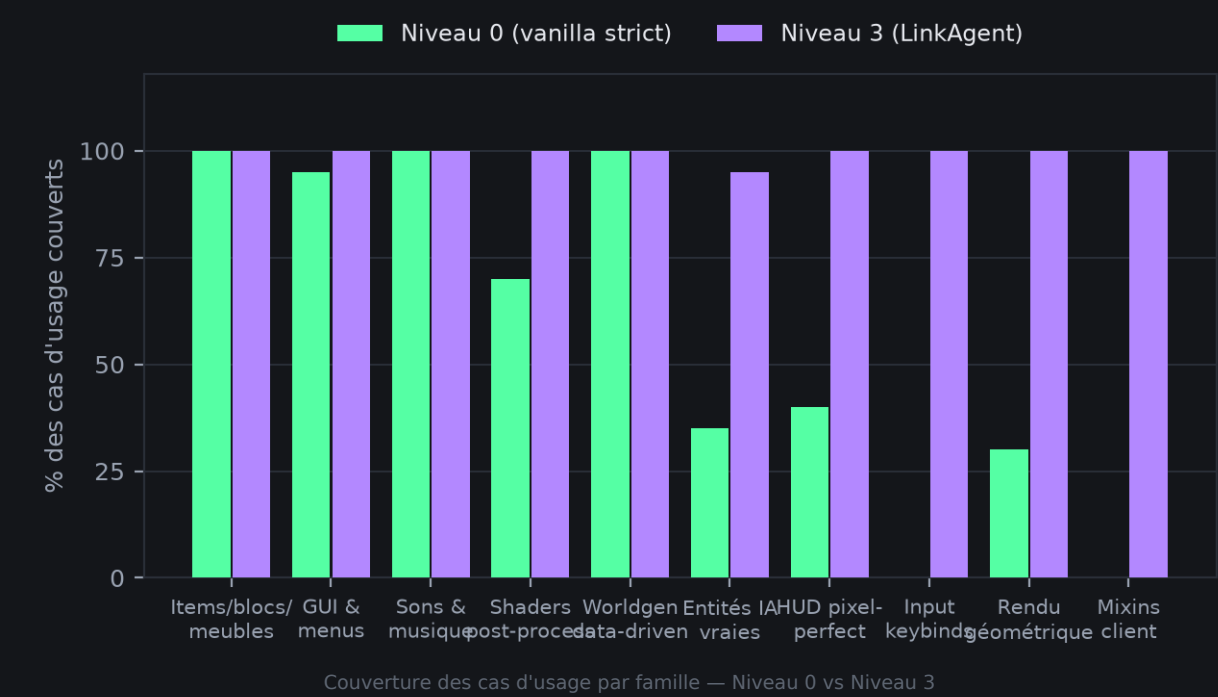
Cookies (5 Ko/clé, autant de clés que voulu) + `ClientboundTransferPacket` : le client se reconnecte automatiquement à un autre serveur en transportant son état (`TransferState`). Une plateforme multi-serveurs (hub → mondes thématiques) fonctionne en Niveau 0 pur, avec progression continue.

2.6 Le mur du Niveau 0 — confirmé dans le code

Besoin	Statut	Preuve source
Exécuter du code client arbitraire	Impossible par construction	Aucun mécanisme ; <code>CustomPayload 1 Mo → if (payload instanceof DiscardedPayload) return; — décodé puis jeté</code>
Nouveau <code>EntityType</code> « vrai » (IA + animations)	Registre client fermé	30 registres synchronisés exactement ; <code>EntityType</code> n'en fait pas partie

Nouveau MenuType / BlockEntityType	• Idem	Idem
Keybinds custom	• Aucun packet	KeyMapping = code client hardcoded
HUD pixel-perfect	• Aucun packet	Workarounds déclaratifs uniquement
Modifier dynamiquement le rendu d'un bloc vanilla	⚡ Statique uniquement	Resource pack = statique ; pas de packet de rendu
Pack > 250 Mo	• Hardcodé	MAX_PACK_SIZE_BYTES = 0xFA000000 (mais stack de packs illimité)
Crash client via faux KnownPacks	☠ Vecteur de DoS, pas une feature	findAndLoadFromResource → IllegalStateException → crash — à documenter comme risque, jamais à exploiter

2.7 Évaluation chiffrée du Niveau 0



En croisant la classification A-I (chapitre 1) avec les mécanismes ci-dessus :

Couverture des cas d'usage de modding ≈ 85 %

✓ Items/blocs/meubles custom

✓ GUI formulaires + menus

✓ Sons/musiques

✓ Shaders post-process

✓ Hologrammes/animations 3D

✓ Worldgen data-driven

- ✓ RPC + état cross-serveur ✓ Bullet-time / cinématiques
- x Keybinds / input custom
- x HUD pixel-perfect
- x Nouvelles entités IA vraies / rendu géométrique custom
- x Toute transformation du client (zoom, HUD tweak, moteur)

Les 15 % restants exigent d'aller chercher le bytecode client — c'est l'objet des chapitres 3 et 4.

Chapitre 3 — Le niveau JVM : classloaders, agents et transformation

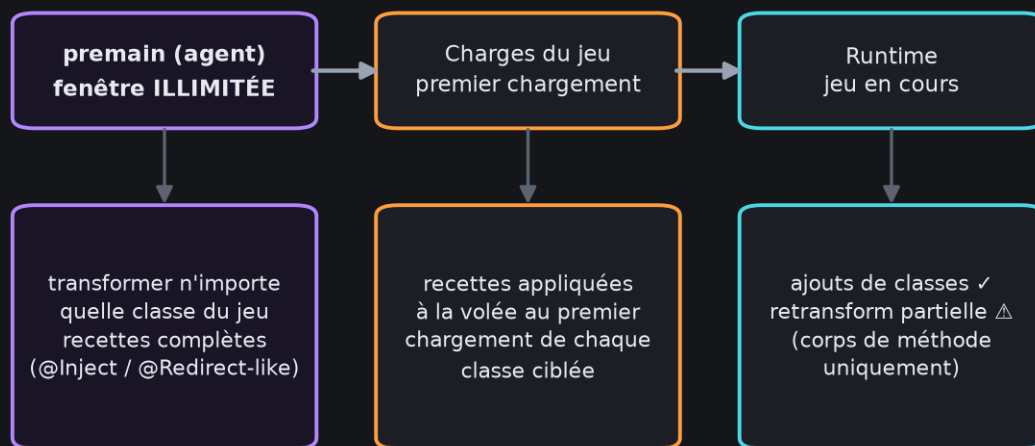
Le mur du Niveau 0 étant structurel (aucun packet ne transporte du code), la question devient : quelle est la plus petite surface client qui permet au serveur d'injecter du comportement ? Ce chapitre descend à la plateforme Java.

3.1 Ce que la JVM offre sous Minecraft

Mécanisme	Ce qu'il permet	Contrainte
<code>ClassLoader.defineClass()</code>	Créer une classe depuis des octets	Nécessite un point d'ancrage déjà présent côté client
<code>URLClassLoader</code> / classloader enfant	Charger des JAR entiers à chaud	Idem
<code>java.lang.instrument (agent)</code>	<code>premain</code> (avant l'app) + <code>ClassFileTransformer</code>	Drapeau <code>-javaagent:</code> au lancement
<code>retransformClasses</code>	Modifier des classes déjà chargées	Pas de nouvelle méthode/champ/superclasse ; ré-écriture de corps de méthode seulement
<code>redefineClasses</code>	Remplacement brutal	Mêmes restrictions + instabilité
JVMTI (natif)	Tout, y compris breakpoints	Hors JVM portable — ignoré ici
Module system (JPMS)	Cloisonnement fort	Minecraft ne l'utilise pas strictement

Fait crucial : Minecraft n'impose aucune de ses propres protections. Le client vanilla tourne sur une JVM standard, sans signature de classes du jeu, sans JPMS strict, sans contrôle de chargement. Qui contrôle le lancement contrôle tout — et « contrôler le lancement » peut signifier `_un unique petit agent générique_`, pas un fork du jeu.

3.2 La fenêtre temporelle de la transformation



Les trois fenêtres de transformation JVM et leurs pouvoirs décroissants

La leçon centrale de Fabric/Mixin : les transformations doivent s'appliquer avant que les classes du jeu ne soient chargées. Après coup, seules les retransformations partielles (corps de méthode) restent possibles.

Lancement JVM

- ↓ premain (agent) ← fenêtre ILLIMITÉE : transformer n'importe quoi
- ↓ classes du jeu chargées
- ↓ main() du jeu
- ↓ runtime ← fenêtre RÉTRÉCIE : addClasses ✓, retransform partielle Δ

Un agent installé au lancement (une seule fois, générique, version-agnostique) ouvre donc la fenêtre complète : il peut enregistrer un transformeur pour les classes du jeu, et le serveur peut lui envoyer à quelles classes appliquer quelles transformations — au premier chargement de chaque classe.

3.3 RemoteClassLoader : l'architecture du chargement à distance

- ```
findClass("com.sdm.mod.HoverBoots")
```
- ↓ miss dans le cache local
  - ↓ requête serveur (canal de transport au choix)
  - ↓ réception bytecode signé
  - ↓ defineClass()
  - ↓ classe disponible dans le jeu

Techniquement trivial pour la JVM — la difficulté n'est pas de charger, c'est d'ancrer le premier point d'entrée. Le client vanilla n'appelle jamais `findClass` sur un serveur distant : il

faut soit un agent au lancement (Niveau 3), soit détourner un mécanisme vanilla existant (chapitre 4, Niveau 1-2).

## 3.4 Le verdict sandbox : il n'y a plus de cage forte en JVM

- JEP 451 : SecurityManager déprécié pour suppression.
- JEP 486 (vérifié sur [openjdk.org](https://openjdk.org/projects/9/roadmap/changes) ce jour) : « Remove the ability to enable the Security Manager when starting the Java runtime... Remove the ability to install a Security Manager while an application is running » — le SecurityManager est mort depuis JDK 24.

Conséquence : le sandboxing mémoire fort d'un code streamé n'existe plus en JVM standard. Les stratégies réalistes, par force décroissante :

1. Process-isolation (processus dédié + IPC) — fort mais architecturalement lourd ;
2. Classloader-isolation + filtrage réseau au niveau processus (paquets firewall du launcher/runtime) ;
3. Permissions déclaratives + signature + réputation — le modèle « plateforme applicative » (comme les stores d'applications) : fort sur la provenance, pas sur l'exécution.

Le rapport retient 3 pour l'architecture cible, avec 1 en option durcissement.

## 3.5 Pourquoi Mixin existe — et ce qu'on peut réduire

Mixin résout trois problèmes : (a) l'obfuscation historique (réglée depuis 1.17+ : les JARs Mojang gardent des noms lisibles), (b) la stabilité des points d'injection entre versions, (c) l'ergonomie des transformations. Une alternative réduite pour SDM :

- Recettes de transformation : descriptions compactes (JSON) — classe cible, méthode, point d'injection, handler module. Équivalent d'un sous-ensemble de Mixin : [@Inject-like](#) (injecter un appel au début/fin de méthode), [@Redirect-like](#) (remplacer un appel), [@ModifyArg-like](#). Pas de [@Shadow/spy](#) complexe : les modules SDM accèdent via reflection helper versionné.
- Ciblage par version : chaque recette référence la version exacte du client (ex. [26.2](#), [26.3-snapshot-7](#)). Le serveur maintient un dossier de recettes par version — c'est lui qui absorbe le coût des mises à jour de mappings, pas l'utilisateur.
- Fallback statique : si aucune recette n'existe pour la version du client, le serveur dégrade proprement (Niveau 0 only) et affiche les limitations via Dialog — jamais de crash.

## 3.6 Précédents hors-Minecraft (chapitre 28 du brief)

| Système     | Précédent applicable                                                                                                                                        |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Garry's Mod | AddCSLuaFile : le serveur envoie du Lua client au joueur qui rejoint — normalisé depuis 15+ ans comme modèle « le serveur est la source du contenu client » |

|                                                  |                                                                                                           |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| FiveM / GTA                                      | Resource streaming : scripts + assets streamés par le serveur, sandbox par ressource, manifest signé      |
| Roblox                                           | Client = runtime universel ; tout le contenu vient des serveurs ; modèle économique de catalogue          |
| Browser / WebAssembly                            | Runtime universel + sandbox + streaming de bytecode compilé — l'analogue exact de « LinkAgent + modules » |
| Steam workshop / jeux live                       | Distribution de contenu sans friction d'installation                                                      |
| Stadia / cloud gaming                            | Thin client ultime — non retenu (bande passante, latence) mais borne théorique de l'axe                   |
| OSGi / plugin IDE                                | Hot-load/unload de bundles, dépendances résolues par la plateforme                                        |
| Nadeshiko (JavaAgent MC expérimental, GitHub ★1) | Preuve de concept que « javaagent + runtime mixins » est un chemin exploré — immature mais validant       |

La thèse SDM n'est donc pas une exotique : c'est le modèle standard des plateformes live (GMod, FiveM, Roblox, web) appliqué à Minecraft, où personne ne l'a encore industrialisé pour Java Edition.

## 3.7 Hot-loading : ce que la JVM permet vraiment en jeu

Scénario du brief — « joueur connecté, serveur active Mod X » :

| Opération                                    | Possible ? | Mécanisme                                                   |
|----------------------------------------------|------------|-------------------------------------------------------------|
| Ajouter des classes                          | ✓          | Nouveau classloader enfant + defineClass                    |
| Charger un module complet                    | ✓          | Idem + résolution de dépendances                            |
| Modifier des classes déjà chargées           | ⚠ Partiel  | retransformClasses : corps de méthode uniquement            |
| Ajouter méthode/champ à une classe existante | • Non      | Limite JVMTI fondamentale                                   |
| Décharger un module                          | ⚠ Partiel  | Classloader orphelin → GC ; classes du jeu liées = résiduel |
| Isoler un module défaillant                  | ✓          | Classloader séparé + kill switch + thread dédié             |

Architecture retenue : modules chargés dans des classloaders dédiés + hooks par interfaces (les classes du jeu appelant des interfaces chargées dans le classloader racine du runtime — pattern déjà éprouvé par les hot-reload IDE/OSGi). Les transformations

profondes restent réservées à la fenêtre pre-main / premier chargement ; à chaud, on compose des ajouts.

## Chapitre 4 — L'escalade : Niveaux 0 → 6, mesurée

Méthode : chaque niveau n'est introduit que lorsque le niveau précédent est prouvé insuffisant. Chaque niveau est mesuré selon les critères du brief : taille, classes modifiées, dépendances, installation, maintenance, compatibilité, sécurité, possibilités obtenues.

### 4.0 Schéma de décision



L'escalade mesurée Niveau 0 → Niveau 6 — chaque palier n'est franchi que sur preuve

NIVEAU 0 — Vanilla strict (85 % des usages)

| mur : code client, types d'entités, input, HUD, rendu profond

↓

NIVEAU 1 — Mécanismes déjà présents poussés à bout (sans rien installer)

| mur : CustomPayload 1 Mo JETÉ (DiscardedPayload) ; aucun hook d'exécution

↓

NIVEAU 2 — Petit bootstrap ponctuel (batch/script) qui ajoute -javaagent

| mur : doit être ré-exécuté par profil ; découverte de l'agent non résolue

↓

NIVEAU 3 — LinkAgent : javaagent générique permanent (~300 Ko, une fois)

| couverture ≈ 98 % ; mur : Mods G « engine rewrite » les plus extrêmes

↓

NIVEAU 4 — Launcher spécialisé (installation complète, MAJ auto du runtime)

| mur : aucun mur technique – mur UX (installer un launcher)

↓

NIVEAU 5/6 – Client patché / fork – REJETÉS : coût de maintenance écrasant,  
écarté du modèle SDM (restent utiles comme fallback de recherche)

## 4.1 Niveau 0 — Vanilla strict

Déjà couvert au chapitre 2. Mesures :

| Critère                      | Valeur                                                        |
|------------------------------|---------------------------------------------------------------|
| Taille client                | 0 Ko                                                          |
| Fichiers / classes modifiées | 0 / 0                                                         |
| Installation                 | Aucune                                                        |
| Maintenance                  | Nulle (tout vit côté serveur)                                 |
| Compatibilité                | Toutes versions (mécanismes protocole stables depuis 1.20.2+) |
| Sécurité                     | Idem vanilla (packs signés optionnels, prompt d'acceptation)  |
| Couverture                   | ≈ 85 % des usages                                             |

## 4.2 Niveau 1 — Mécanismes déjà présents, poussés à bout

Sans rien installer, un serveur peut encore tenter :

- Stéganographie du handshake : hostname 255 chars pré-auth (pattern Velocity/Bungee) — utile pour router/négocier, pas pour exécuter.
- CustomPayload 1 Mo S→C : décodé par le client... puis jeté (`DiscardedPayload`). Bande passante/CPU uniquement. Confirmer comme vecteur mort pour vanilla pur.
- Faux KnownPacks : crash le client (DoS). Documenté comme attaque, jamais comme capacité.
- Exfiltration C→S discrète : `SetTestBlockPacket.message` (32 767 chars, op-level requis).

Verdict : le Niveau 1 ajoute des canaux périphériques, aucune capacité d'exécution. Le mur est confirmé : sans ancre d'exécution, rien ne traverse.

## 4.3 Niveau 2 — Bootstrap ponctuel (sans launcher)

L'utilisateur (ou un script fourni par son serveur) exécute une fois un installeur léger qui :

1. Copie `linkagent.jar` (~300 Ko) dans un dossier ;
2. Enregistre `-javaagent:linkagent.jar` sur le profil Minecraft (fichier `launcher_profiles.json` / profils tiers) ;
3. Ne touche à rien d'autre — le client Minecraft reste le binaire officiel intact.

Mesures :

| Critère              | Valeur                                                                                                                  |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Taille               | ~300 Ko                                                                                                                 |
| Classes MC modifiées | 0 (l'agent s'attache au lancement, ne modifie pas les fichiers)                                                         |
| Installation         | 1 double-clic + valider (les launchers officiels/tiers relisent le profil)                                              |
| Maintenance          | Faible ; l'agent est générique (aucune version MC dedans)                                                               |
| Risques              | Les launchers réécrivent parfois les profils → réinstallation ponctuelle ; l'argument JVM non standard peut être ignoré |
| Couverture           | ≈ 98 % (voir Niveau 3)                                                                                                  |

Limite du Niveau 2 : la découverte. Un joueur qui découvre un serveur SDM doit savoir qu'il faut le bootstrap. C'est le Niveau 3/4 qui rend la découverte automatique.

## 4.4 NIVEAU 3 — LinkAgent : l'agent générique permanent

Le résultat central du rapport. Un javaagent unique, installé une fois (via Niveau 2 ou 4), qui transforme définitivement le client en terminal universel :

| Critère                  | Valeur                                                                                                 |
|--------------------------|--------------------------------------------------------------------------------------------------------|
| Taille                   | ~300 Ko (ASM ~130 Ko + codec + crypto + API cœur)                                                      |
| Contenu                  | premain, ClassFileTransformer, RemoteClassLoader, cache signé, négociation SDMP, permissions           |
| Ce qu'il ne contient PAS | Aucun mapping, aucune logique de mod, aucun contenu — tout vient du serveur                            |
| Installation             | Une fois, par bootstrap ou launcher                                                                    |
| Compatibilité            | Toutes versions MC : les recettes/mappings ciblent des versions précises et sont streamées par serveur |



|                   |                                                                            |
|-------------------|----------------------------------------------------------------------------|
| <b>Sécurité</b>   | <b>Signature Ed25519 obligatoire, permissions déclaratives, révocation</b> |
| <b>Couverture</b> | <b>≈ 98 % — tout sauf les réécritures moteur les plus radicales</b>        |

Responsabilités de l'agent :

1. Écouter le handshake (plugin message login / cookie / hostname) et négocier les capacités SDMP ;
2. Télécharger et vérifier modules + recettes + packs (signature Ed25519, cache disque versionné) ;
3. Appliquer les recettes aux classes du jeu au premier chargement (fenêtre complète) ;
4. Charger les modules dans des classloaders enfants avec permissions ;
5. Exposer l'API runtime (events, rendu, input, HUD, registres virtuels) aux modules ;
6. Vieillir dignement : version MC inconnue → dégradation Niveau 0 propre, jamais de crash.

Pourquoi ~300 Ko suffisent : l'agent embarque ASM (transformation), un client HTTP(S) (transport), Ed25519 (JCA natif), et ~40 classes runtime. Tout le savoir version-spécifique vit côté serveur — c'est LE renversement : le coût des mises à jour Minecraft passe du joueur au serveur.

## 4.5 Niveau 4 — Launcher spécialisé

Un launcher (ou extension d'un launcher tiers : Prism, MultiMC, Modrinth App, ATLauncher...) intègre le LinkAgent nativement : installation zéro-clic, découverte automatique des serveurs SDM, cache partagé inter-serveurs, mises à jour du runtime.

- Avantages : UX maximale, distribution simplifiée, brand sécurité (signatures vérifiées par le launcher).
- Inconvénient : dépendance à l'écosystème launcher — contre l'objectif d'indépendance maximale. À traiter comme accélérateur optionnel, pas comme socle.

## 4.6 Niveaux 5-6 — Client patché / fork

Rejetés comme architecture cible : chaque version Minecraft = re-patcher, re-distribuer (zones grises légales), re-maintenir un fork. Le LinkAgent rend ces niveaux inutiles — ils ne subsistent que comme chantiers de recherche isolés (ex. études de rendu).

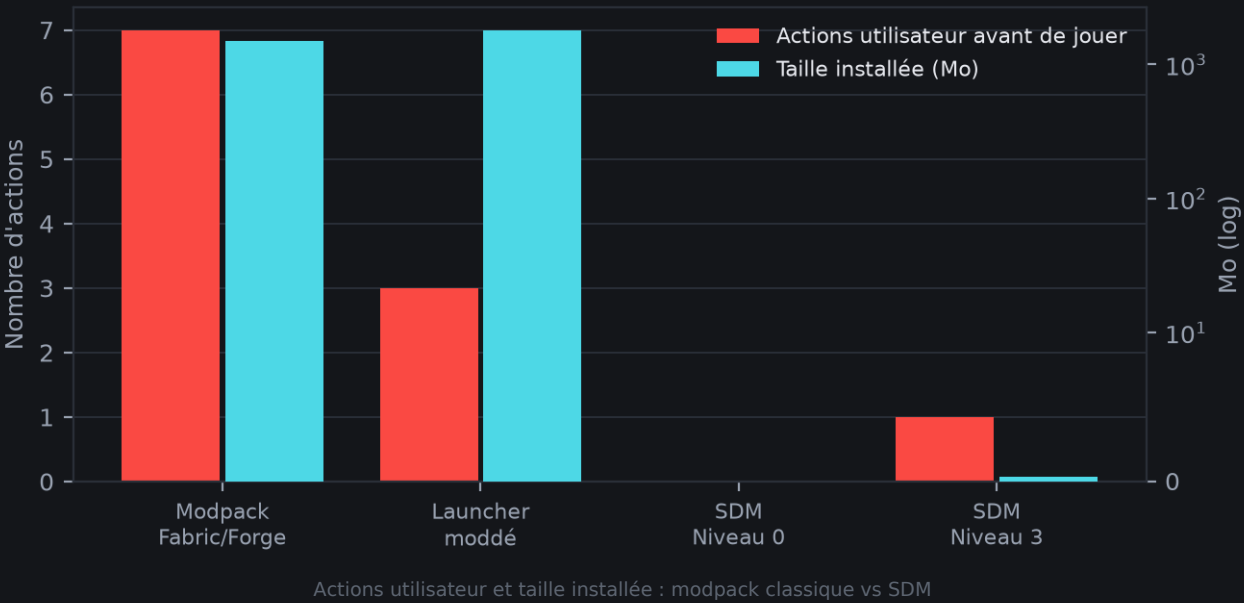
## 4.7 Le tableau d'or : capacités × niveaux (réponse au §25 du brief)

Légende : ✓ natif · ~ approximable · ✗ bloqué

| Capacité                              | N0 | N3 (agent) | Note                                    |
|---------------------------------------|----|------------|-----------------------------------------|
| Item custom (apparence /comportement) | ✓  | ✓          | CMD + ITEM_MODEL ; comportement serveur |
| Bloc custom statique/meuble           | ✓  | ✓          | Virtualisation polymère + Display       |

|                                 |   |       |                                                             |
|---------------------------------|---|-------|-------------------------------------------------------------|
| Vraie entité IA custom          | ~ | ✓     | N0 : marionnette serveur ; N3 : type virtuel + rendu module |
| GUI formulaires/menus           | ✓ | ✓     | Dialog + fake menus                                         |
| HUD pixel-perfect               | ~ | ✓     | N0 : bossbar/scoreboard ; N3 : module overlay               |
| Rendu custom (géométrie/shader) | ~ | ✓     | N0 : PostEffects + Display ; N3 : module rendu              |
| Input keybind                   | ✗ | ✓     | Module input via agent                                      |
| Mixins client (zoom, tweak)     | ✗ | ✓     | Recettes de transformation                                  |
| Custom packets                  | ~ | ✓     | N0 : canaux NBT ; N3 : canal SDMP natif                     |
| Worldgen custom                 | ✓ | ✓     | Data-driven + registres synchronisés                        |
| Hot-load module en jeu          | ~ | ✓     | N0 : packs/Dialogs à chaud ; N3 : classes à chaud           |
| Multi-versions                  | ✓ | ✓     | Recettes ciblées par version                                |
| Compat Fabric/Forge directe     | ✗ | ✗ → ~ | Adaptateur partiel (chapitre 8)                             |

## 4.8 Comparaison des modèles d'installation (§17 du brief)



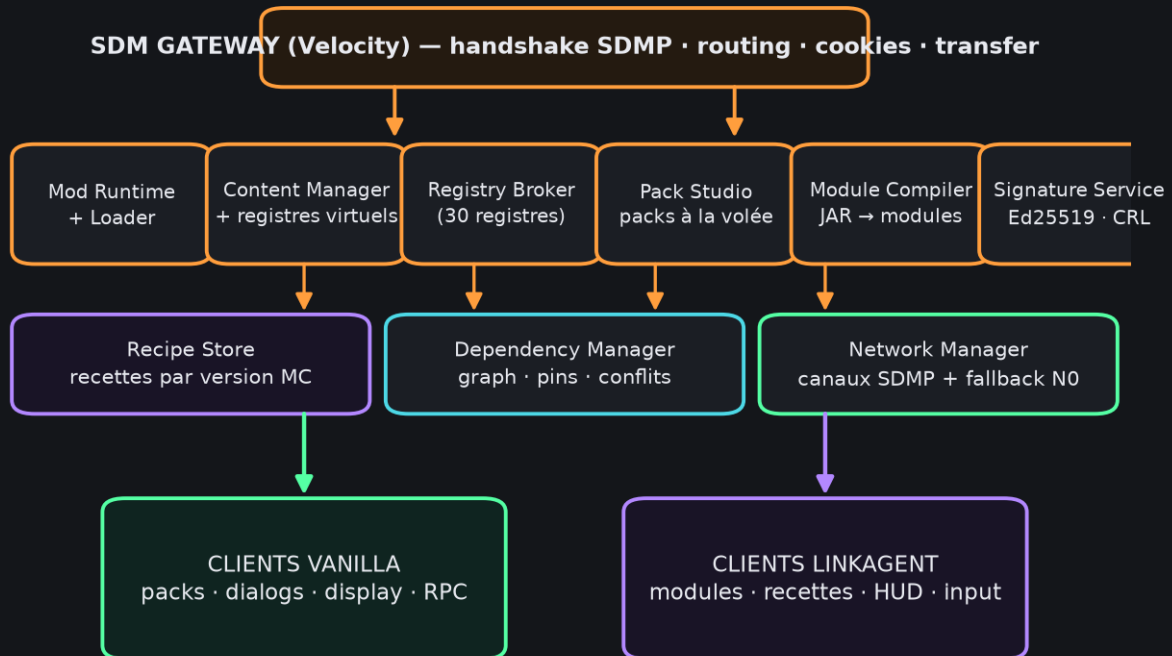
| Modèle                                 | Simplicité | Sécurité | Feasibilité               | UX                             |
|----------------------------------------|------------|----------|---------------------------|--------------------------------|
| A — zéro installation (Niveau 0)       | ★★★★★      | ★★★★★    | Prouvée                   | « Lancer, rejoindre, jouer »   |
| B — bouton unique (bootstrap Niveau 2) | ★★★★☆      | ★★★★☆    | Prouvée (fichiers profil) | 1 clic une fois                |
| C — launcher dédié (Niveau 4)          | ★★★☆☆      | ★★★★★    | Prouvée                   | 1 installation d'app           |
| D — association automatique            | ★★★★☆      | ★★★☆☆    | Dépend launchers tiers    | Idéal si adopté                |
| E — bootstrap permanent (LinkAgent)    | ★★★★★      | ★★★★☆    | Cible SDM                 | 1 clic une fois, tous serveurs |

Verdict : E (via B pour l'installation initiale) est l'optimum — zéro action pour tous les serveurs SDM après la première installation.

## Chapitre 5 — Architecture serveur : la plateforme de modding

Le serveur devient la plateforme. Ce chapitre définit l'architecture complète du côté serveur, pièce par pièce.

### 5.1 Vue d'ensemble



Architecture de la plateforme SDM côté serveur

#### PLATEFORME SDM (serveur)

```

| SDM Gateway (proxy Velocity) — handshake SDMP, routing, |
| cookies, transfer inter-serveurs |
|
| SDM Server Core (plugin/fork Canvas-Paper) |
| — Mod Runtime — chargement modules serveur |
| — Mod Loader — discovery, graph de dépendances |
| — Content Manager — registres virtuels (items/blocs) |
| — Registry Broker — sync des 30 registres vanilla |
| — Virtualization Layer — polymère-like : blocs virtuels |
| — Network Manager — canaux SDMP, fallback Niveau 0 |
| — Pack Studio — génération resource packs à la volée |

```

```

	(textures/models/sounds/shaders/fonts)
	— Dependency Manager — résolution + pins + conflits
	— Module Compiler — JAR serveur → modules client
	(extraction side-client + recettes par version MC)
	— Recipe Store — mixins ciblés par version
	— Signature Service — Ed25519, manifest, révocation
	— Telemetry — erreurs client agrégées
————— Compiler backends —————	
▼ ▼
Clients vanilla (Niveau 0) Clients LinkAgent (Niveau 3)
packs + dialogs + display modules + recettes + packs

```

## 5.2 Les couches logicielles

### Gateway (Velocity)

Le point d'entrée réseau gère le handshake SDMP avant même le login : lecture du hostname (255 chars) pour routing/capabilities, cookies d'identification, transfer inter-serveurs. C'est aussi lui qui détermine le niveau du client (vanilla / agent) et compose la réponse adaptée.

### Server Core

Un plugin (voire une fork légère) sur le serveur de jeu (Canvas/Paper/Folia) qui implémente le runtime :

- Mod Runtime : classloader hiérarchique, lifecycle (load → start → stop → unload), isolation par module ;
- Registres virtuels : le Content Manager alloue des IDs virtuels (custom blocks sur supports vanilla), le Registry Broker synchronise les 30 registres vanilla quand utile, la Virtualization Layer produit les équivalents visuels ;
- Pack Studio : compile les assets des modules en resource packs, hash + signature, push à chaud ; canalisé par version MC (format pack 95 / data 115 en 26.3-snapshot-7) ;
- Module Compiler : chaque mod écrit une fois (code commun) est dérivé en : module serveur (logique), module client (présentation), recettes (transformations), packs (assets), data (loot/worldgen). Le serveur est l'usine à modules.

### Recipe Store

La banque de recettes de transformation, indexée par version MC exacte. Chaque entrée : classe cible (nom Mojang-mapped), méthode, injection point, handler module, hash de la classe cible attendue (échec propre si mismatch). C'est ici que vit le coût des mises à jour Minecraft — assumé par l'opérateur de plateforme, pas par les joueurs.

## Signature Service

Tous les artefacts streamés (modules, recettes, packs) sont signés Ed25519. Le manifeste décrit modules, versions, dépendances, permissions demandées, hashes. La révocation (CRL/OCSP-like) permet de tuer un module compromis.

## 5.3 Le cycle de vie d'un module serveur→client

1. Développeur publie → SDM DevKit (API universelle + adapters) → plateforme
2. Plateforme compile → module serveur + module client + recettes + packs + data
3. Joueur rejoint → Gateway handshake : niveau détecté (N0 ou N3)
- 4a. N0: packs streamés, dialogs, display entities, canaux NBT
- 4b. N3: manifest signé → agent vérifie → télécharge → applique recettes  
(classes jeu non encore chargées) → charge modules → ACK runtime
5. En jeu → events serveur → modules via SDMP ; hot-swap possible

## 5.4 API universelle (§24 du brief)

L'API cible est indépendante du loader :

ServerModAPI (le contrat)

Blocks · Items · Entities · World · Networking · Events · GUI

Rendering · Input · Commands · Registries · Data · Resources · Permissions

Adapters (implémentations)

Vanilla-protocol (Niveau 0) — dialogs/packs/display/CMD

LinkAgent (Niveau 3) — modules SDMP complets

Fabric / Forge / NeoForge — exécution native côté serveur

Un mod écrit contre ServerModAPI fonctionne sur toute plateforme ayant un adapter — y compris un serveur Fabric qui stream vers clients vanilla via l'adapter Vanilla-protocol. C'est la réplique du rôle de Architecture API (93 M downloads) mais étendu à la distribution.

## 5.5 Mod-per-world, mod-per-player, hot operations (§14-15 du brief)

- Mods par monde : le Gateway route par serveur/monde ; chaque monde stream son manifeste. Les cookies transportent la progression entre mondes.

- Mods par joueur : le serveur maintient un profil de capacités par joueur (permissions modules) — un joueur VIP voit des modules optionnels qu'un autre ne voit pas.
- Hot-load : activation d'un module pour joueurs connectés = push manifeste delta → l'agent charge dans un classloader neuf (additions de classes toujours possibles à chaud) → ACK. Les transformations profondes nécessitent la re-entrée PLAY→CONFIG (mécanisme vanilla) pour re-synchroniser — le joueur reste connecté.
- Hot-unload : désactivation + isolation (le module cesse d'être appelé, classloader orphelin candidat GC). La JVM ne garantit pas l'unload immédiat — documenté comme limite.

## 5.6 Servir plusieurs versions Minecraft (§23 du brief)

Le Recipe Store + Pack Studio versionnent tout : recettes par version MC exacte, packs par format de pack. Le module client (code) est compilé par la plateforme contre les mappings de chaque version cible — le serveur maintient une matrice de build. L'agent, lui, ne sait rien des versions. Résultat : un joueur en 26.2 et un joueur en 26.3 sur le même serveur reçoivent chacun les artefacts de leur version — la promesse multi-versions est tenue par construction, au prix de la maintenance serveur des mappings.

## 5.7 Performance (§20 du brief)

| Dimension         | Analyse                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Client            | Agent ~300 Ko + modules chargés à la demande ; RAM maîtrisée par lazy-loading                                                               |
| Réseau            | Cache disque signé (hashes) — un module n'est téléchargé qu'une fois ; packs différentiels                                                  |
| CPU client        | Modules simples (HUD, hooks events) négligeables ; rendu custom = budget GPU normal                                                         |
| Serveur           | Virtualisation + génération de packs = coûts amortis (compilation une fois, cache par version)                                              |
| Latence           | Rien d'interactive ne traverse le réseau au-delà de l'existant : input local → module local, events serveur → modules via SDMP (asynchrone) |
| Premier lancement | Manifest + modules d'un modpack léger ≈ 20-80 Mo ; 1-10 s sur fibre                                                                         |
| Comparaison       | Modpack classique = 0,5-2 Go installés localement, avant même de jouer                                                                      |

## 5.8 Matrice des capacités (§9 du brief)

| Capacité | Fabric | Forge | NeoForge | SDM N0       | SDM N3 |
|----------|--------|-------|----------|--------------|--------|
| Blocks   | ✓      | ✓     | ✓        | ✓ (virtuels) | ✓      |

|                 |                      |         |         |                   |                    |
|-----------------|----------------------|---------|---------|-------------------|--------------------|
| Items           | ✓                    | ✓       | ✓       | ✓                 | ✓                  |
| Entities        | ✓                    | ✓       | ✓       | ~ (visuelles)     | ✓                  |
| GUI             | ✓                    | ✓       | ✓       | ✓ (dialogs/menus) | ✓                  |
| Rendering       | ✓                    | ✓       | ✓       | ~ (post/GPU)      | ✓                  |
| Networking      | ✓                    | ✓       | ★       | ✓                 | ✓                  |
| Registry        | ✓                    | ✓       | ★       | ✓ (30 sync)       | ✓                  |
| Worldgen        | ✓                    | ✓       | ✓       | ✓                 | ✓                  |
| Events          | ✓                    | ✓       | ★       | ✓ (serveur)       | ✓ (client+serveur) |
| Mixins client   | ✓                    | partiel | partiel | ✗                 | ✓ (recettes)       |
| Client hooks    | ✓                    | ✓       | ★       | ✗                 | ✓                  |
| Bytecode        | ✓                    | ✓       | ★       | ✗                 | ✓ (agent)          |
| Dynamic loading | ~                    | ~       | ~       | ~ (packs)         | ✓                  |
| Distribution    | ✗ (install manuelle) | ✗       | ✗       | ✓                 | ✓                  |

La ligne décisive : Distribution — la seule capacité qu'aucun loader n'offre et que SDM apporte nativement.



## Chapitre 6 — SDMP : le protocole et le LinkAgent

## 6.1 Philosophie du protocole

SDMP (Server-Driven Modding Protocol) superpose deux couches :

Protocole Minecraft (handshake/login/config/play)

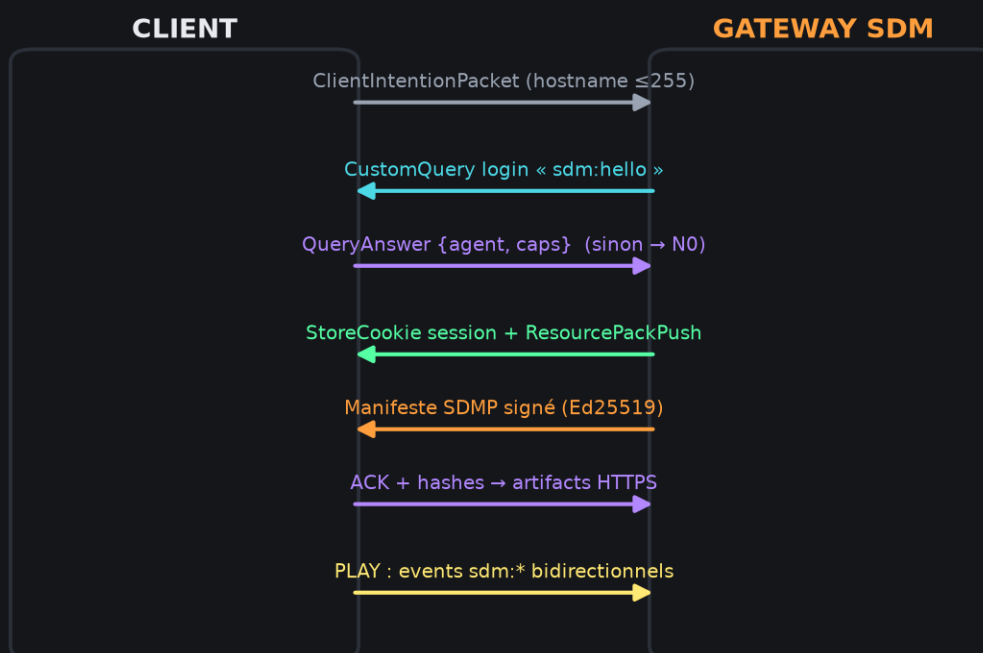
+

SDMP (négociation, manifestes, modules, recettes, events, lifecycle)

Deux modes de transport selon le niveau du client :

- Transport vanilla (Niveau 0) : SDMP s'appuie sur les canaux existants — resource pack prompt, cookies, CustomClickAction, CustomQuery (login plugin messaging), hostname handshake. Le client vanilla « parle SDMP » sans le savoir.
- Transport agent (Niveau 3) : un canal binaire dédié sur CustomPayload (channel `sdm:main`, 1 Mo/packet S→C) + HTTP(S) parallèle pour les gros artefacts (le client agent ne télécharge pas les modules par le socket de jeu — il utilise HTTPS avec tokens jetables).

## 6.2 Séquence de négociation



Négociation SDMP — du handshake au canal d'événements, avec bascule N0 automatique

Client Gateway SDM

|— ClientIntentionPacket (hostname 255) —→| routing + hint SDM

| | (velocity-style)

|← CustomQuery (login) « sdm:hello » —|

```

|→ QueryAnswer {agentVersion, caps} →| (agent only; vanilla = timeout → N0)

| |

| ... login / configuration ... |

|← StoreCookie « sdm:session » (5 Ko) →| token session, version plateforme

|← ResourcePackPush (packs base signés) →| (les DEUX modes)

|← [agent] Manifeste SDMP signé →| modules, recettes, permissions

|→ [agent] ACK + hash set →|

|← [agent] Artifacts (HTTPS 206 range) →| modules JAR + recettes JSON

| |

| ... play ... |

|↔ Events SDMP (channel sdm:main) →| hooks runtime bidirectionnels

```

Décision de niveau avant le jeu : l'absence de réponse à `sdm:hello` en login/config classe le client N0 — le serveur ne stream pas de modules, il compose l'expérience Niveau 0. Jamais d'erreur visible.

## 6.3 Manifeste

Exemple de manifeste signé (JSON condensé) :

```

{

 "platform": "coco-station",

 "platformVersion": "1.7.0",

 "mcVersions": ["26.2", "26.3-snapshot-7"],

 "modules": [

 {

 "id": "cocomagic",

 "version": "3.2.1",

 "perms": ["render:overlay", "input:keybinds", "net:sdm"],

 "artifacts": {

 "server": "cocomagic-server-3.2.1.jar",

 "client": "cocomagic-client-3.2.1+26.2.jar",

 "recipes": "cocomagic-recipes-26.2.json",

 "pack": "cocomagic-pack-3.2.1.zip"
 }
 }
]
}

```

```

},

"depends": [{"id": "sdm-api", "range": ">=2.0"}],

"sha256": "...", "sigEd25519": "..."

}

],

"revocation": "https://registry.sdm.dev/crl",

"minAgent": "1.0.0"

}

...

```

## 6.4 Le LinkAgent : anatomie (~300 Ko)

LinkAgent

- ├─ premain — s'attache avant le jeu, enregistre le transformer
- ├─ TransformerGateway — applique les recettes au premier chargement
  - | (ClassFileTransformer) des classes ; no-op tant que rien n'est reçu
- ├─ RemoteClassLoader (racine) — charge les modules, parent = classloader jeu
- ├─ CacheManager — disque : modules + packs par (serveur, version)
- ├─ Verify — Ed25519 + SHA-256 + pinning clé plateforme
- ├─ PermissionBroker — applique le manifeste perms (UI de consentement)
  - | └─ ConsentUI — écran vanilla-like (via Dialog ou screen injecté)
- ├─ SDMPClient — transport jeu + HTTPS artifacts
- └─ RuntimeBridge — expose l'API aux modules (events, rendu, input, HUD)

Aucune de ces classes ne contient le moindre mapping Minecraft. L'agent est version-agnostique : la classe cible d'une recette est désignée par son nom Mojang-mapped (« net.minecraft.client.renderer.GameRenderer »), lu via l'API de réflexion de l'agent. Une version MC inconnue → recettes absentes → Niveau 0 fallback → le jeu marche quand même.

## 6.5 Events runtime (extraits)

| Event               | Sens | Charge                        |
|---------------------|------|-------------------------------|
| sdm.hello / sdm.ack | C→S  | version agent, caps, hash set |

|                                        |     |                                              |
|----------------------------------------|-----|----------------------------------------------|
| <code>sdm.manifest</code>              | S→C | manifeste signé (delta si cache)             |
| <code>sdm.module.load / .unload</code> | S→C | hot-load en jeu                              |
| <code>sdm.event.*</code>               | S→C | events jeu encapsulés vers modules           |
| <code>sdm.input.*</code>               | C→S | keybinds module → serveur (si perm accordée) |
| <code>sdm.render.*</code>              | S→C | ordres de rendu overlay                      |
| <code>sdm.error</code>                 | C→S | rapport d'erreur module (telemetry)          |

## 6.6 Séquence hot-load (§15 du brief)

```

Joueur en jeu, serveur active « CocoMagic »

↓ sdm.module.load {id, version, artifacts, sig}

↓ agent: manifest delta vérifié (Ed25519, perms déjà consenties → silencieux)

↓ nouvelles perms ? → ConsentUI (sinon transparent)

↓ téléchargement HTTPS (cache check par hash)

↓ nouveau ModuleClassLoader (additions de classes = toujours OK à chaud)

↓ si recettes nécessaires sur classes déjà chargées → demande serveur

de re-entrée PLAY→CONFIG (StartConfiguration) → retransform → retour au jeu

↓ module ACK → visible pour le joueur

```

Le pire cas (retransform needed) reste transparent : le joueur voit un écran de rechargement de ~2-5 s, équivalent à un changement de resource pack — jamais un crash.

## 6.7 Dégradation gracieuse — la règle d'or

Toute la conception obéit à une règle : un client sans agent, ou un agent sans recettes pour sa version, doit toujours pouvoir jouer en Niveau 0. Le serveur porte l'intelligence de dégradation : même contenu, exprimé en dialogs/packs/display si les modules sont indisponibles. Cette règle garantit la coexistence vanilla/agent sur le même serveur et élimine le syndrome « serveur inaccessible car mod manquant ».

## Chapitre 7 — Sécurité : le modèle de confiance

L'architecture introduit un serveur qui fournit du code au client. C'est le problème majeur identifié par le brief — traité ici au même niveau que la faisabilité.

### 7.1 La menace dans la forme la plus claire

Un joueur qui rejoint un serveur malveillant avec le LinkAgent accepte implicitement d'exécuter du code signé par ce serveur. Sans garde-fous, c'est un RCE-as-a-service. Le modèle doit tenir compte de :

1. RCE serveur→client : le module streamé est du code natif JVM complet (accès disque, réseau, processus — JEP 486 a tué la sandbox forte) ;
2. Vol de session Minecraft : un module peut lire les tokens de session ;
3. Persistance : cache disque = code qui reste ;
4. Exfiltration : modules réseau non restreints ;
5. Supply chain : module populaire compromis ;
6. DoS client : manifests géants, recettes cassées, packs corrompus.

### 7.2 Le modèle retenu : signature + permissions + consentement + révocation



Chaîne de confiance SDM — signature, consentement, moindre privilège, révocation

C'est le modèle « plateforme applicative » (stores d'apps, FiveM Signed Scripts, code-signing web) — le seul réaliste en JVM moderne :

#### Serveur

↓ Manifeste signé Ed25519 (clé plateforme)

↓ Registre de modules (provenance : plateforme, auteur signant, ou serveur local)

#### User Trust

```

↓ Premier serveur SDM d'une nouvelle clé → écran de consentement

↓ (empreinte clé + permissions demandées, style « installer une app »)

Module Download

↓ Vérif Ed25519 + SHA-256 par artefact + pinning

Sandbox relatif / Runtime

↓ PermissionBroker : perms accordées = seules actives

↓ Modules serveur-local : sandbox réseau restreint par défaut

Execution

↓ Telemetry erreurs + kill switch runtime (CRL)

```

## Permissions déclaratives (extrait)

| Permission                  | Effet                                            |
|-----------------------------|--------------------------------------------------|
| <code>net:sdm</code>        | Parler SDMP au serveur courant                   |
| <code>net:external</code>   | Réseau sortant arbitraire (cartes, maps...)      |
| <code>fs:cache</code>       | Écriture cache dédiée                            |
| <code>fs:profile</code>     | Lecture profil/screenshots (consentement dédié)  |
| <code>render:overlay</code> | HUD/overlay                                      |
| <code>input:keybinds</code> | Enregistrer des keybinds                         |
| <code>sys:process</code>    | Sous-processus (jamais par défaut — écran dédié) |

## Règles structurelles

- Jamais de code non signé exécuté ; signature invalide = refus + rapport.
- Principe de moindre privilège : un module n'obtient que ses perms déclarées ; le PermissionBroker bloque le reste au niveau du RuntimeBridge.
- ToS Minecraft : la distribution du client Minecraft lui-même reste interdite ; SDM ne distribue jamais de fichiers du jeu — l'agent s'attache au client officiel déjà possédé par le joueur (usage guidelines Mojang : « we reserve the right... » — la prudence impose de ne pas redistribuer d'assets Mojang, ce que SDM évite par construction).
- Kill switch : la CRL permet à la plateforme de révoquer un module sur tous les clients en quelques secondes.
- Réputation : agrégation telemetry (crash rate, reports) exposée dans l'écran de consentement.

## 7.3 Ce que ce modèle ne protège pas (honnêteté technique)

- Un module signé malveillant dont les perms incluent `net:external` peut exfiltrer des données que le jeu rend accessibles. Mitigations : consentement granulaire, réputation, révocation — pas de prévention absolue.
- L'isolation mémoire n'existe pas (JEP 486). Un module compromis au sein d'une plateforme négligente = code complet sur la machine. La défense est organisationnelle (signatures, audit) — identique à « installer un mod CurseForge », mais avec révocabilité.
- Le vecteur « premier join » : si l'utilisateur accepte aveuglément. UX de consentement claire = la meilleure arme (empreinte de clé, perms, réputation, source).

## 7.4 Comparaison avec les modèles connus

| Système             | Modèle                          | Leçon pour SDM                                                                              |
|---------------------|---------------------------------|---------------------------------------------------------------------------------------------|
| Garry's Mod / FiveM | Lua sandbox / scripts signés    | Le streaming serveur→client est socialement accepté depuis 15 ans                           |
| Roblox              | Sandbox Luau stricte            | Preuve qu'un runtime universel à sandbox fort marche à centaines de millions d'utilisateurs |
| Navigateurs         | Sandbox multi-process + origine | Le niveau or ; hors de portée JVM, mais inspire le consentement par origine (serveur)       |
| Fabric/CurseForge   | « télécharge et espère »        | Aucune révocation, aucune perm — SDM est strictement mieux outillé                          |
| Java Web Start      | Sandbox dépréciée puis morte    | La sandbox JVM légère est une impasse (JEP 451/486)                                         |

## 7.5 Option durcissement : l'isolation processus

Pour les plateformes exigeantes (compétition, prix réels), un mode durci où l'agent lance les modules dans un processus satellite (JVM séparée, IPC mémoire partagée + sockets locales) : un module compromis ne touche pas le processus jeu, ni la session. Coût : complexité runtime + latence IPC. Positionné comme option entreprise, pas comme socle.

## Chapitre 8 — Compatibilité écosystème, cas de test, PoC et roadmap

## 8.1 Compatibilité écosystème (§8 du brief) — les 5 niveaux

| Niveau                            | Description                                                 | Faisabilité                                | Effort      |
|-----------------------------------|-------------------------------------------------------------|--------------------------------------------|-------------|
| 1. Reproduire les fonctionnalités | Réécrire les features en ServerModAPI                       | ✓ Immédiat                                 | Par mod     |
| 2. API compatible                 | Offrir les surfaces Fabric-like (events, registries)        | ✓                                          | Moyen       |
| 3. Adaptateur partiel             | Charger des parties de mods (logique serveur, data, assets) | ✓                                          | Moyen-élevé |
| 4. Chargement direct              | Exécuter des mods Fabric/Forge tels quels                   | Partiel — mods server-side only            | Élevé       |
| 5. Runtime écosystème complet     | Faire tourner la quasi-totalité du catalogue                | ⚠ Théorique —                              | Très élevé  |
|                                   |                                                             | les mixins client ciblent le client local, |             |
|                                   |                                                             | pas un runtime streamé                     |             |

Lecture honnête : les niveaux 1-3 sont réalistes et couvrent la majorité de la valeur. Le niveau 4 marche pour les mods purement serveur (le module serveur SDM peut embarquer et exécuter le JAR Fabric server-side via adapter). Le niveau 5 bute sur une vérité structurelle : les mods à mixins client profonds (Sodium, Iris...) supposent un client moddé local — SDM ne les servira jamais sans les réécrire, sauf à ce que le LinkAgent implémente un loader complet (ce qui reviendrait à réinstaller Fabric universellement : possible techniquement, contraire à l'esprit minimal).

Hydraulic comme précédent : permettre à des clients Bedrock de rejoindre des serveurs Java moddés (traduction d'architecture) prouve que la traduction de contenu custom entre runtimes hétérogènes est faisable — GeyserMC l'exploite déjà pour Geyser (5748★). SDM est la généralisation Java↔Java-vanilla de ce pattern.

## 8.2 Les 12 cas de test (§26 du brief)

| # | Cas         | Niveau requis | Implémentation SDM                                        |
|---|-------------|---------------|-----------------------------------------------------------|
| 1 | Item simple | 0             | CMD + ITEM_MODEL + pack ; comportement via events serveur |



|    |                                           |     |                                                                              |
|----|-------------------------------------------|-----|------------------------------------------------------------------------------|
| 2  | Bloc                                      | 0   | Virtualisation (support vanilla + modèle) ; hitbox serveur                   |
| 3  | Entité                                    | 0-3 | N0 : Display puppet serveur-piloté ; N3 : entity type virtuel + rendu module |
| 4  | GUI custom                                | 0   | Dialog + inputs + CustomClickAction RPC                                      |
| 5  | Packet custom                             | 0   | Canaux NBT (CustomClickAction 32 Ko / cookies 5 Ko) ; N3 : sdm:main natif    |
| 6  | Gameplay complexe (jobs/économie)         | 0   | Serveur pur + dialogs + packs                                                |
|    |                                           |     |                                                                              |
| 7  | Worldgen                                  | 0   | Data packs + registres synchronisés (BIOME, DIMENSION_TYPE)                  |
| 8  | Rendu                                     | 0-3 | N0 : PostEffects + Display ; N3 : module rendu custom                        |
| 9  | Input (keybind dash)                      | 3   | Module input (perm input:keybinds) + event serveur                           |
| 10 | Modification profonde client              | 3   | Recettes de transformation versionnées                                       |
| 11 | Mod à mixins denses (zoom + HUD tweak)    | 3   | Recettes équivalentes + module présentation                                  |
| 12 | Mod complexe Forge/NeoForge (Create-like) | 0-3 | Niveau 1-2 compat : réécriture ServerModAPI ; N0 : sous-ensemble visuel      |

## 8.3 PoC — démonstration de faisabilité minimale

Objectif : prouver les trois piliers en 4 semaines de dev.

### P1 — Niveau 0 pur (semaine 1)

Serveur Paper/Canvas + plugin prototype :

- 1 item custom complet (papier → épée visuelle via CMD) avec pack streamé ;
- 1 dialog de quête avec TextInput + action CustomAll → RPC serveur ;
- 1 hologramme animé (TextDisplay + interpolation) ;

- 1 cookie de progression + transfer vers un second serveur.

Preuve : client vanilla officiel, zéro installation, expérience « mod-like ».

## P2 — LinkAgent minimal (semaines 2-3)

- premain + transformer + 1 recette (injecter un hook dans `GameRenderer` pour un HUD overlay simple) ;
- transport : CustomPayload `sdm:main` + HTTPS pour le module ;
- manifeste signé Ed25519 + cache + vérification.

Preuve : le serveur a ajouté un HUD custom à un client officiel sans que l'utilisateur n'installe un mod.

## P3 — Hot-load (semaine 4)

- Activation d'un module en jeu (classloader neuf + ACK) ;
- Re-entrée `PLAY`→`CONFIG` pour une recette additionnelle.

Preuve : la promesse hot-loading tient sur les deux fenêtres JVM.

## Critères de succès

Client vanilla strict joue P1 sans rien installer ; client agent reçoit P2+P3 sans crash ; dégradation `N3`→`N0` vérifiée (agent désactivé = le serveur repasse en mode `dialogs/packs`).

## 8.4 Roadmap 11 phases (§31.K du brief)

| Phase                         | Contenu                                                                                | Durée indicative               |
|-------------------------------|----------------------------------------------------------------------------------------|--------------------------------|
| 1. Vanilla research           | Audit protocole complet par version (décompilation, catalogue des mécanismes, limites) | 2-4 sem (déjà largement faite) |
| 2. Protocol research          | Spéc SDMP v0 (hello/manifest/events), codecs, canaux vanilla vs agent                  | 2-3 sem                        |
| 3. Server prototype           | Plugin plateforme : registres virtuels, Pack Studio, RPC N0                            | 4-6 sem                        |
| 4. Vanilla-client experiments | P1 complet + tests multi-versions + telemetry                                          | 3-4 sem                        |
| 5. Minimal bootstrap          | Installeur 1-clc multi-launchers (profils JVM args)                                    | 2-3 sem                        |
| 6. Remote runtime             | LinkAgent v1 : premain, transformer, cache, verify                                     | 4-6 sem                        |
| 7. Dynamic code loading       | RemoteClassLoader + hot-load + re-entrée CONFIG                                        | 3-4 sem                        |
| 8. Mod API                    | ServerModAPI + DevKit + docs + 5 mods exemple                                          | 6-8 sem                        |

|                                             |                                                                              |                |
|---------------------------------------------|------------------------------------------------------------------------------|----------------|
| <b>9. Fabric compatibility</b>              | <b>Adapter Fabric côté serveur +<br/>extraction assets/data</b>              | <b>4-6 sem</b> |
| <b>10. Forge/NeoForge<br/>compatibility</b> | <b>Idem + étude de faisabilité<br/>niveau 4</b>                              | <b>6-8 sem</b> |
| <b>11. Production architecture</b>          | <b>Gateway multi-serveurs, registry<br/>signé, CRL, telemetry, hardening</b> | <b>continu</b> |

Total vers une bêta publique : ~9-12 mois à petite équipe (2-3 devs), en supposant les phases 1-2 déjà acquises.

## 8.5 Analyse des limites — tableau final (§25)

Fonctionnalité → Serveur seul ? → Vanilla client ? → Bootstrap ? → Runtime ? → Client patch ?

Item custom oui oui non non non

Bloc custom oui oui (virtuel) non non non

Entité IA custom oui (logique) puppet seulement non oui (rendu) non

GUI formulaires oui oui non non non

HUD pixel-perfect non approx (bars) non oui non

Keybind custom non non non oui non

Rendu géométrique non partiel (post) non oui non

Deep mixin client non non non oui (recettes) équiv.

Worldgen oui oui non non non

Code arbitraire non non – oui (signé) équiv.

## 8.6 Positionnement par rapport aux modèles existants

| Modèle                       | Rapport à SDM                                                                                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fabric/Forge/NeoForge</b> | <b>Complémentaires côté serveur ; rivaux côté distribution. SDM ne les remplace pas sur le desktop modding profond — il leur substitue un canal de distribution pour le grand public</b> |
| <b>Polymer/PolyFactory</b>   | <b>Preuve de concept du Niveau 0 à l'échelle — SDM industrialise ce que Polymer fait en framework</b>                                                                                    |
| <b>Geyser/Hydraulic</b>      | <b>Preuves de la translation d'architecture hétérogène — SDM généralise Java→Java-vanilla</b>                                                                                            |

|                          |                                                                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ViaVersion</b>        | <b>Précédent de multi-versions par translation serveur — même philosophie d'absorption du coût côté serveur</b>                                       |
| <b>Simple Voice Chat</b> | <b>Preuve qu'un mod à composante client peut être adopté massivement — mais exige toujours installation : SDM supprime précisément cette friction</b> |

## 8.7 Conclusion générale

La réponse à la double question finale du brief :

1. Oui — Minecraft peut devenir une plateforme où le client vanilla sert de base d'exécution et où le serveur streame les capacités : à 85 % dès aujourd'hui sans rien installer, et à ~98 % avec un agent générique de ~300 Ko installé une seule fois. Les preuves existent déjà éparpillées (Polymer, Geyser, Dialog system, resource pack hot-swap) ; il manque la couche d'orchestration — protocole, runtime, sécurité, plateforme — que ce rapport spécifie.
2. La plus petite modification client optimale n'est ni un launcher, ni un fork, ni un mod : c'est un javaagent générique version-agnostique, car il est le seul composant qui peut transformer le client en terminal universel tout en gardant le binaire Minecraft officiel intact, et en laissant au serveur 100 % du coût des versions et du contenu.